Funções em JavaScript

Manual pedagógico passo a passo

Do primeiro contacto com uma função até à interação com botões e campos HTML.

Formador: Marcos Alvarães

Sintaxe de uma função

Uma função é um conjunto de instruções identificado por um nome. Pode receber valores, executar operações e devolver um resultado.

Estrutura de uma função

```
function nomeDaFuncao(parametro1, parametro2) {
    // Instruções que a função executa

    return resultado;
}
```

Explicação de cada elemento

Elemento	Significado
function	Palavra reservada para declarar uma função.
nomeDaFuncao	Nome que atribuímos à função.
( )	Parênteses que podem conter parâmetros.
parametro1, parametro2	Valores que a função pode receber.
{ }	Chavetas que delimitam o corpo da função.
return	Devolve um valor e termina a execução da função.

Importante: uma função não é obrigada a receber parâmetros nem a utilizar return.

Exemplo simples — Função sem parâmetros

```
function saudacao() {
    console.log("Bom dia!");
}

// Chamar a função
saudacao();
```

Resultado na consola: Bom dia!

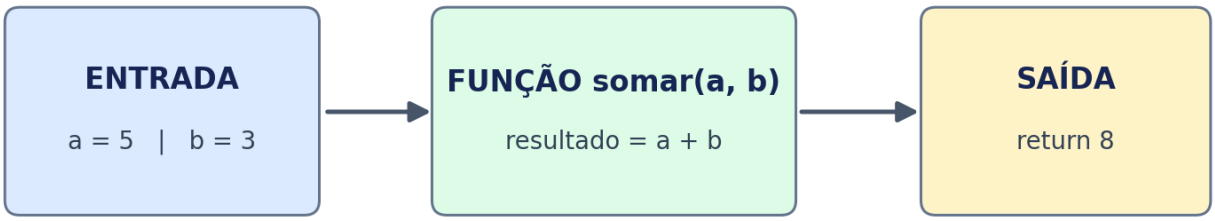
A função só é executada quando escrevemos saudacao(). A declaração define a função; a chamada executa as instruções.

Exemplo — Função com parâmetros

```
function somar(a, b) {
    let resultado = a + b;
    return resultado;
}

// Chamar a função
let total = somar(5, 3);

console.log(total);
```



Como funciona a função somar()?

Chamamos a função com somar(5, 3).
O parâmetro a recebe 5 e o parâmetro b recebe 3.
A função calcula 5 + 3.
O return devolve 8.
A variável total guarda o valor 8.

Regra fundamental

É importante distinguir três operações diferentes:

Declarar a função:

```
function somar(a, b) {
    return a + b;
}
```

Chamar a função:

```
somar(5, 3);
```

Guardar e apresentar o resultado:

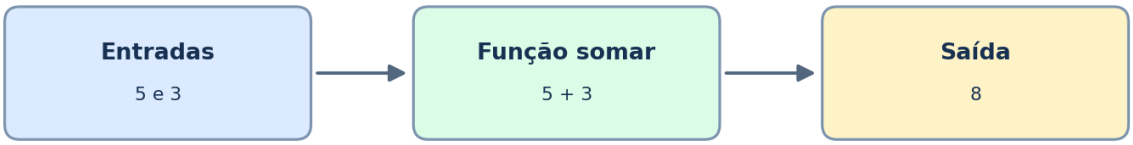
```
let resultado = somar(5, 3);
console.log(resultado);
```

IDEIA PRINCIPAL

Uma função pode receber valores (parâmetros), executar instruções e devolver um resultado (return).

Esta é a sintaxe recomendada para iniciar a aprendizagem, antes de introduzir funções anónimas, arrow functions ou outras formas de declarar funções em JavaScript.

1. O que é uma função?

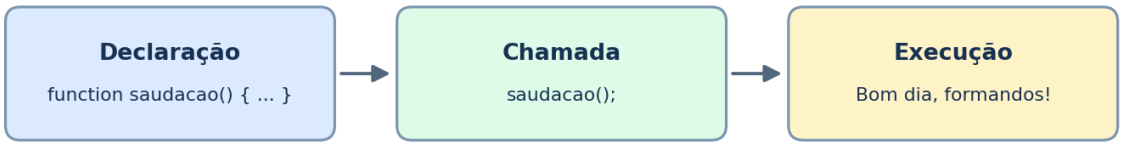


Uma função pode receber dados, executar instruções e produzir um resultado.

Uma função é um bloco de instruções com um nome que pode ser executado sempre que necessário. Use a analogia de uma calculadora: recebe números, realiza uma operação e pode devolver um resultado. Nem todas as funções precisam de receber dados ou devolver valores.

Esquema para escrever no quadro: ENTRADAS → FUNÇÃO → RESULTADO. Exemplo: 5 e 3 → somar → 8.

2. Criar e chamar a primeira função



Definir uma função não a executa: é necessário chamá-la.

Passo 1 — Função sem parâmetros

Objetivo: Perceber a diferença entre definir e executar.

```
function saudacao() {
    console.log("Bom dia, formandos!");
}

saudacao();
```

Explicação: function inicia a declaração; saudacao é o nome; os parênteses estão vazios; as chavetas delimitam as instruções. A última linha chama a função.

Pergunta de verificação: O que acontece se apagarmos saudacao();? (A função fica definida, mas não é executada.)

Passo 2 — Reutilizar a mesma função

Objetivo: Executar várias vezes sem duplicar o bloco de instruções.

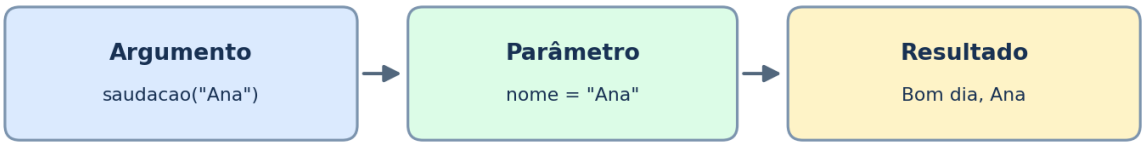
```
function saudacao() {
    console.log("Bom dia, formandos!");
}

saudacao();
saudacao();
saudacao();
```

Explicação: A mensagem aparece três vezes, apesar de o código da função ter sido escrito apenas uma vez.

Pergunta de verificação: Quantas mensagens aparecem? (Três.)

3. Introduzir parâmetros



O argumento enviado na chamada é recebido pelo parâmetro da função.

Passo 3 — Receber um nome

Objetivo: Compreender a função dos parâmetros.

```
function saudacao(nome) {
    console.log("Bom dia, " + nome);
}

saudacao("Ana");
saudacao("Pedro");
saudacao("Maria");
```

Explicação: nome é o parâmetro (variável local da função). "Ana", "Pedro" e "Maria" são argumentos, isto é, os valores enviados em cada chamada.

Pergunta de verificação: O que aparece se escrevermos saudacao("João");? (Bom dia, João.)

Passo 4 — Dois parâmetros

Objetivo: Efetuar uma operação com dois valores.

```
function somar(a, b) {
```

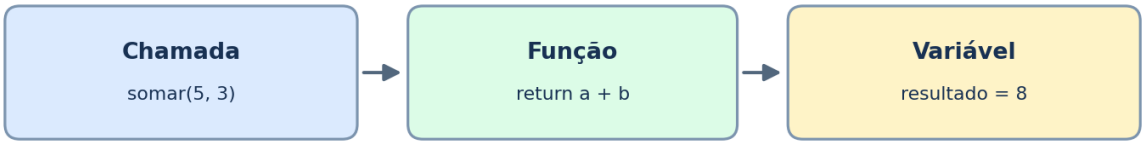
```
    console.log(a + b);
}

somar(5, 3);
somar(10, 20);
```

Explicação: a e b são parâmetros; os argumentos 5 e 3 são usados na primeira chamada. A consola mostra 8 e 30.

Pergunta de verificação: O que acontece com somar(7, 2);? (Mostra 9.)

4. Devolver resultados com return



return devolve o valor calculado para o local onde a função foi chamada.

Passo 5 — Guardar o resultado

Objetivo: Distinguir console.log de return.

```
function somar(a, b) {
    return a + b;
}

let resultado = somar(5, 3);
console.log(resultado);
```

Explicação: return devolve o valor 8 à expressão que chamou a função. let resultado guarda esse valor; console.log apenas o apresenta na consola.

Pergunta de verificação: Se não escrevermos console.log(resultado), o cálculo continua a ser realizado? (Sim.)

console.log()	return
Mostra informação na consola.	Devolve um valor a quem chamou a função.
Útil para observar e testar.	Útil para guardar e reutilizar o resultado.
Não substitui o valor devolvido pela função.	Termina a execução da função naquele ponto.

Passo 6 — Reutilizar valores devolvidos

Objetivo: Combinar resultados de duas chamadas.

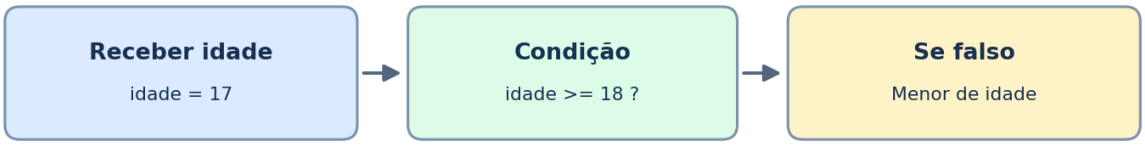
```
function somar(a, b) {
    return a + b;
}

let resultado1 = somar(5, 3);
let resultado2 = somar(10, 20);
let total = resultado1 + resultado2;
console.log(total);
```

Explicação: A primeira chamada devolve 8, a segunda devolve 30 e o total apresentado é 38.

Pergunta de verificação: Podemos usar somar(2, 4) diretamente dentro de outra expressão? (Sim.)

5. Tomar decisões dentro de uma função



O if/else permite escolher o resultado em função de uma condição.

Passo 7 — Verificar maioridade

Objetivo: Reutilizar conhecimentos de if/else.

```
function verificarMaioridade(idade) {
    if (idade >= 18) {
        return "É maior de idade.";
    } else {
        return "É menor de idade.";
    }
}

console.log(verificarMaioridade(12));
console.log(verificarMaioridade(20));
```

Explicação: A função recebe a idade, compara-a com 18 e devolve uma das mensagens. Mostre que o return pode devolver texto, não apenas números.

Pergunta de verificação: Qual é o resultado para 18? (É maior de idade.)

6. Ligar funções ao HTML



O evento de clique inicia o cálculo; textContent apresenta o resultado na página.

Nesta etapa, recorde que HTML constrói a interface e JavaScript trata a interação. Separe os ficheiros para os formandos identificarem claramente as responsabilidades.

Passo 8 — Criar index.html

```
<!DOCTYPE html>
<html lang="pt-PT">
<head>
    <meta charset="UTF-8">
    <title>Funções em JavaScript</title>
</head>
<body>
    <h1>Somar dois números</h1>
    <input type="number" id="num1" placeholder="Primeiro número">
    <input type="number" id="num2" placeholder="Segundo número">
    <button id="botao">Somar</button>
    <p id="resultado"></p>
    <script src="script.js"></script>
</body>
</html>
```

Explicar: os inputs recebem valores; o botão desencadeia uma ação; o parágrafo mostra o resultado; o atributo id permite identificar cada elemento a partir do JavaScript.

Passo 9 — Criar script.js

```
function somar(a, b) {
    return a + b;
}

document.getElementById("botao").addEventListener("click", function() {
    let campo1 = document.getElementById("num1").value;
    let campo2 = document.getElementById("num2").value;

    if (campo1.trim() === "" || campo2.trim() === "") {
        document.getElementById("resultado").textContent =
            "Preencha os dois campos!";
        return;
    }
```

```
    let a = Number(campo1);
    let b = Number(campo2);
    let resultado = somar(a, b);

    document.getElementById("resultado").textContent =
        "Resultado: " + resultado;
});
```

Explicação guiada do código

Instrução	O que explicar
getElementById("botao")	Seleciona o botão através do seu id.
addEventListener("click", ...)	Executa as instruções quando há um clique.
.value	Lê o conteúdo do campo, inicialmente como texto.
.trim() === ""	Deteta um campo vazio ou só com espaços.
return; dentro do evento	Interrompe o tratamento do clique se faltarem dados.
Number(campo1)	Converte o texto em número para somar numericamente.
somar(a, b)	Chama a função de cálculo.
textContent	Escreve o resultado no parágrafo da página.

Nota: a função anónima passada a addEventListener pode ser apresentada apenas como as instruções a executar ao clicar. Não é necessário introduzir já funções arrow ou conceitos avançados.

Passo 10 — Testar

Entrada	Resultado esperado
5 e 3	Resultado: 8
10 e 20	Resultado: 30
-2 e 7	Resultado: 5
Primeiro campo vazio	Preencha os dois campos!
Segundo campo vazio	Preencha os dois campos!
0 e 4	Resultado: 4

Saliente que zero é um valor válido; um campo vazio não deve ser tratado como zero. Para exercícios introdutórios, a validação apresentada é suficiente para os testes propostos com campos numéricos.

7. Exercícios autónomos

1 — Mensagem

Enunciado: Criar uma função mensagem() que apresente «Estou a aprender JavaScript!» na consola.

Solução de referência:

```
function mensagem() {
    console.log("Estou a aprender JavaScript!");
}
mensagem();
```

2 — Mostrar nome

Enunciado: Criar mostrarNome(nome) que apresente o nome recebido.

Solução de referência:

```
function mostrarNome(nome) {
    console.log("Nome: " + nome);
}
mostrarNome("Ana");
```

3 — Multiplicação

Enunciado: Criar multiplicar(a, b) que devolva o produto de dois números.

Solução de referência:

```
function multiplicar(a, b) {
    return a * b;
}
console.log(multiplicar(4, 5));
```

4 — Quadrado

Enunciado: Criar calcularQuadrado(numero) que devolva o quadrado do número.

Solução de referência:

```
function calcularQuadrado(numero) {
    return numero * numero;
}
console.log(calcularQuadrado(6));
```

8. Avaliação formativa

responder oralmente ou por escrito:

☐ Qual é a diferença entre declarar e chamar uma função?

☐ Para que servem os parâmetros?

☐ Qual é a diferença entre parâmetro e argumento?

☐ Qual é a diferença entre console.log() e return?

☐ Para que serve Number() ao ler um input?

☐ Porque verificamos campos vazios antes da conversão?

9. Erros frequentes e correções

Erro frequente	Como explicar/corrigir
Criar uma função e esquecer a chamada	A declaração não executa automaticamente o bloco.
Escrever somar sem parênteses para a chamar	Usar somar(5, 3) para efetuar a chamada.
Confundir return com console.log	return devolve; console.log apresenta.
Somar strings de inputs	Converter com Number() antes de somar.
Campo vazio interpretado como zero	Validar o texto antes de Number().
id do HTML diferente do JavaScript	Confirmar que os identificadores coincidem.